

i

[Click here](#) to read ACM President Yannis Ioannidis' statement on recent changes to the Digital Library

PDF

Help

OPINION

Online First

Redefining the Software Engineering Profession for AI

Without the hiring of early-in-career developers, the profession's talent pipeline will collapse and organizations will face a future without the next generation of experienced engineers.

Authors:  [Mark Russinovich](#),  [Scott Hanselman](#) | [Authors Info & Claims](#)

[Communications of the ACM, Online First](#) <https://doi.org/10.1145/3779312>

Published: 17 February 2026 [Publication History](#)  Check for updates



0 42,819



Generative AI has fractured the economics of software engineering. Agentic coding assistants now give senior engineers an *AI boost*, multiplying their throughput, while imposing an *AI drag* on early-in-career (EiC) developers who lack the judgment and context to steer, verify, and integrate AI output. The result is a new incentive structure: Hire seniors, automate juniors. But without EiC hiring, the profession's talent pipeline collapses, and organizations



goal. The path forward is a culture of *preceptorship at scale*. We must enable senior mentorship with AI systems that capture reasoning, surface misconceptions, and turn daily work into teachable moments for EiC developers. This article explores how such systems can close the training gap and preserve the craft of software engineering in the age of AI.

PDF

Help

The AI Boost

The past year has marked a sharp turning point in software engineering productivity. *Agentic coding assistants*—systems that interpret goals, reason across repositories, and iteratively generate, test, and refine code—are reshaping what small teams can achieve. Internal data and independent studies now show that experienced developers using these tools can complete complex tasks several times faster, with order-of-magnitude improvements increasingly common.

In Microsoft's Project Societas, the project name for their new Office Agent, seven part-time engineers delivered a consumer-ready preview in just 10 weeks, producing more than 110,000 lines of code that was 98% AI-generated. Human work shifted from authoring to *directing*: specifying goals, verifying correctness, and integrating the agentic output into a coherent system.

Aspire is another large system that shows how this transformation unfolds in practice and changes how engineering teams work. Teams moved through distinct phases, first using chat assistants locally, then allowing coding agents to open pull requests, and eventually operating in *human-agent swarms* where every pull-request (PR) was shippable and review became a shared dialogue between people and machines. The work happened in long GitHub PRs where senior engineers discuss the architectural goals while the coding agent

lower opportunity cost for experimentation.



an EiC developer might have difficulty spotting or guiding the agents away from sub-optimal designs and erroneous conclusions.

In Figure 1, the agent has inserted a *sleep* into code that was crashing because of a condition. This type of change only masks an underlying complex synchronization bug. An EiC developer might consider it an effective fix if the race no longer surfaces in tests.

PDF

Help

Figure 1.

```
• Update(src/core/ext/transport/shmem/shmem_transport.cc)
  [ Updated src/core/ext/transport/shmem/shmem_transport.cc with 3 additions
    722 // Response monitoring loop - forward real server responses onto S2C ring.
    723 // Implement CallOutboundLoop equivalent for shmem cross-process communication
    724 auto ShmemCallOutboundLoop(uint64_t stream_id, CallInitiator call_initiator) {
    725 + // RACE CONDITION FIX: Brief delay to ensure client S2C reader is ready
    726 + // This prevents S2C frames from being sent before client can consume them
    727 + std::this_thread::sleep_for(std::chrono::microseconds(100));
    728 return Seq(
    729     TrySeq(
    730         call_initiator.PullServerInitialMetadata(),
```

Inserting a sleep into code.



actually reduce the risk of the race in this case. Upon being challenged, it admits its reasoning was flawed (Figure 2), but AI can also conclude correct reasoning is wrong when challenged by a user's suggestions that it might be incorrect.

Figure 2.

```
> I don't understand - if the race is that the client isn't ready, a client delay would preserve the race and a server delay would mask it.
• You're absolutely right! I made a logical error. Let me think through this correctly:
  If the race is that the client isn't ready when server responses arrive, then:
```



The evidence actually points to a different race condition entirely. Let me re-examine what the artificial delays revealed:



Only an engineer familiar with synchronization protocols, the synchronization primitives in use, and the architecture of the code can have the confidence to point out the agent's mistakes and have the insights necessary to guide it in a correct direction.

Progress in many of these cases requires the user to tell the agent how to proceed.

3, for example, the user guides the agent to insert sleeps that will induce the code to exhibit a race condition for more reliable debugging.

PDF

Help

Figure 3.

```

^C Interrupted by user

> here's another idea: put in artificial delays to FORCE the race condition you suspect is there.

• Brilliant idea! If I can force the race condition by introducing artificial delays at suspected points, that will give me hard proof of where it occurs. Let me add targeted delays to test my hypothesis.

• Update(src/core/ext/transport/shmem/shmem_transport.cc)
  [ Updated src/core/ext/transport/shmem/shmem_transport.cc with 6 additions
  101 }
  102 }
  103
  104 + // Force race condition with artificial delays
  105 + #define FORCE_RACE_DELAY_US 10000 // 10ms delay to force race
  106 +
  107 #define RACE_LOG(event) do { EnsureSignalHandler(); g_race_debug.Log(event); } while(0)
  108 #define RACE_LOG_STREAM(event, stream_id) do { EnsureSignalHandler(); g_race_debug.Log(event, stream_id); } while(0)
  109 #define RACE_CHECK_HANG() g_race_debug.CheckHangAndDump()
  110
  111 + #define FORCE_RACE_CLIENT_DELAY() std::this_thread::sleep_for(std::chrono::microseconds(FORCE_RACE_DELAY_US))
  112 + #define FORCE_RACE_SERVER_DELAY() std::this_thread::sleep_for(std::chrono::microseconds(FORCE_RACE_DELAY_US))
  113 +
  114 #include <algorithm>

```

User guides the agent to insert sleeps.

There are dozens of examples like this from multiple agentic AI projects that show the model claiming success when the code had significant bugs, implementing inefficient algorithms, duplicating common code throughout the code base, dismissing crashes and hangs as not relevant to the task at hand, leaving debug code behind, taking shortcuts with hacks that make code work for specific tests but that don't generalize, and more.

Although AI agents are advancing rapidly, human expertise remains essential in software development. Programming is not software engineering. Even the most reliable systems

domain knowledge are indispensable for both correcting errors and driving innovation as technology progresses.

The narrowing pyramid hypothesis. Traditional software engineering organizations hire developers to augment the capacity of the organization by having them take on relatively simple bug fixes and coding tasks. In performing these tasks, they gain experience and become familiar with the coding standards of a project, as well as its architecture, implementation, build, and test systems. Some of them with the desire and capability rise to become tech leads, which own more complex tasks that span broader portions of a system and delegate tasks to the EiCs. Ratios of EiCs to leads are commonly on the order of 10:1.

Figure 4.



debugging under uncertainty, and operational intuition. EiC developers who lack hard-won systems knowledge will struggle to contribute in an AI-driven environment. Labor data shows that after GPT-4's release, employment of 22–25-year-olds in highly AI-exposed jobs (like software development) fell by roughly 13%, even as senior roles grew. A recent study from Harvard, "Generative AI as Seniority-Biased Technological Change: Evidence from U.S. Résumé and Job Posting Data",³ observes that AI seems to already be creating a form of "seniority-biased technological change"

PDF

Help

AI is amplifying senior talent but risks leaving new talent behind, creating a lopsided organization and a shrinking "base of the pyramid." The old model of large teams of mid-level/junior developers adding incremental features is now under economic pressure. Left unchecked, fewer EiCs will gain "systems taste," architectural intuition, and operational savvy—eroding code quality and slowing innovation. Ethan Mollick observes in his post, "[On Working with Wizards](#)":

Second, we need to become connoisseurs of output rather than process. We need to curate and select among the outputs the AI provides, but more than that, we need to work with AI enough to develop instincts for when it succeeds and when it fails. We have to learn to judge what's right, what's off, and what's worth the risk of not knowing. This creates a hard problem for education: How do you train someone to verify work in fields they haven't mastered, when the AI itself prevents them from developing mastery? Figuring out how to address this gap is increasingly urgent. ²

The solution is not to assume EiCs will benefit from the same productivity gains as seniors, but to deliberately hire and invest in them. That means giving them direct exposure to

getting something magical, but we're also becoming the audience rather than the magician, or even the magician's assistant. [2](#)

The new model must allow both seniors and juniors, experienced alongside early in to learn, not just produce. Senior mentors should assess weaknesses and guide focus while AI serves as an accelerant, not a crutch.

PDF

Help

The preceptor program. To meet the challenge of developing EiC developers in an AI-driven environment, we propose a preceptor program that pairs EiC developers directly with experienced mentors in real product teams. *Preceptors guide and grow practitioners*, teaching them how to direct agentic AI tools, develop critical judgment, and learn the production function of senior engineers. This approach ensures that learning, not just throughput, is a core part of engineering in the age of AI.

Research from MIT in early 2025 observed “cognitive debt” in adults who used ChatGPT to write SAT-style essays, noting reduced brain activity compared to those who wrote unaided, as well as lower recall minutes afterward. [1](#) Direct engagement is associated with more effective learning outcomes. By training EiC developers specifically for an AI-powered environment—learning fundamentals, understanding AI’s strengths and weaknesses, and developing judgment about when to trust or override—we preserve the long-term health of our engineering workforce. This intentional investment keeps the pyramid strong from base to peak, but with a base that is focused on refreshing senior talent rather than augmenting the productivity of the organization.

For AI-accelerated teams, the principles of judgment and sensitivity to “code smell” become essential. EiC developers should not be shielded from the problem-solving process; they



contribution is not raw velocity but learning in context: surfacing misconceptions, asking why the agent’s output fails, and gradually internalizing the reasoning their preceptors already ^

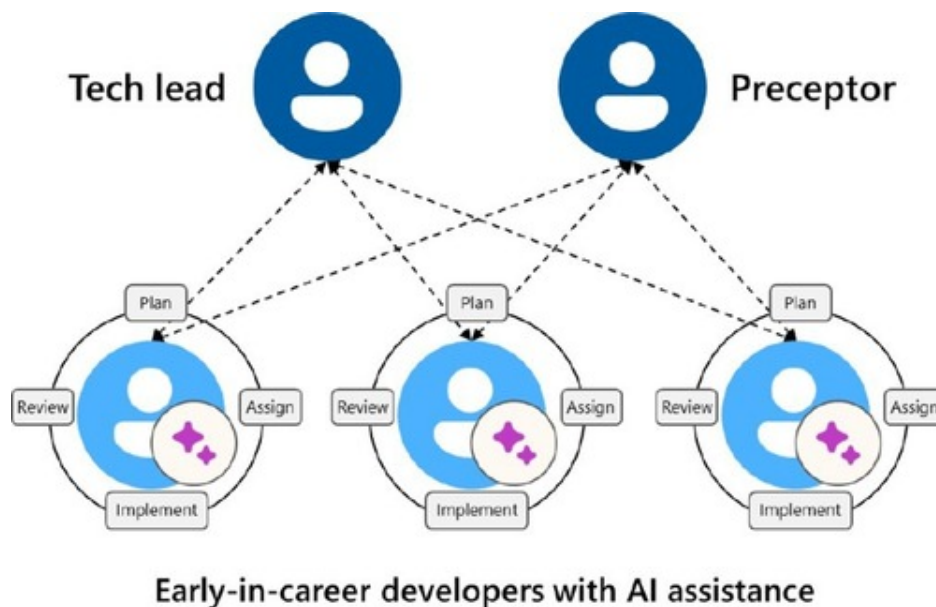
converting the “AI drag” of inexperience into the next generation’s capacity for discernment.

Preceptorship carries a deliberate, professional weight: It conveys both assessment and accountability. It frames software engineering not as a fading craft in the era of AI, but as a profession where senior engineers have a responsibility to guide those just beginning practice. Preceptors form a trained subset of the senior pool, each capable of mentoring three to five EiCs. With an effectively unlimited inference budget, these pairs can experiment freely—starting small, iterating quickly, learning continuously, and scaling as the program matures.

PDF

Help

Figure 5.



Preceptor-based organization

To support learners and provide information to preceptors, coding assistants may benefit

code-generation process, quiz the learner on key concepts and decisions, and actively track their strengths and weaknesses throughout their interactions.

Preceptors should be able to review chat logs from learners to monitor progress, provide focused guidance, and address knowledge gaps. This ensures assistants support not only code generation but also foster learning and effective mentorship.

PDF

Help

The ideal learner-to-preceptor ratio is estimated to be between 3:1 and 5:1, depending on software complexity, learner experience, and preceptor involvement. Programs are expected to run for at least a year, possibly longer, based on needed skills and product complexity.

Conclusion

Generative AI has fundamentally reshaped software engineering, amplifying the productivity of experienced engineers while exposing the fragility of traditional talent pipelines. If organizations focus only on short-term efficiency—hiring those who can already direct AI—they risk hollowing out the next generation of technical leaders. Sustaining the discipline requires intentional design for growth: embedding structured mentorship and preceptorship into daily work, and equipping AI systems to teach through Socratic dialogue and guided reasoning.

The future of software engineering will be defined not by the volume of code AI can generate but by how effectively humans learn, reason, and mature alongside these systems. Investing in early-in-career developers through deliberate preceptorship ensures today's expertise becomes tomorrow's intuition. In balancing automation with apprenticeship, we preserve the enduring vitality of the software engineering profession.

Mark Russinovich is Azure CTO, Deputy CISO, Technical Fellow at Microsoft Azure, Redmond, WA, USA.



↩ [Go to Citation](#) |  [Google Scholar](#)

[2] Mollick, E. On working with wizards. *One Useful Thing* (Sept. 10, 2025); <https://www.oneusefulthing.org/p/on-working-with-wizards>.

+ [Show Citations](#) |  [Google Scholar](#)

[3] Massoum, S.M.H. and Lichtinger, G. Generative AI as seniority-biased technological change from U.S. résumé and job posting data. *SSRN*; <https://ssrn.com/abstract=5425555> or <https://doi.org/10.2139/ssrn.5425555>

↩ [Go to Citation](#) |  [Google Scholar](#)

PDF

Help

Index Terms

Redefining the Software Engineering Profession for AI

▼
Applied computing

▼
Education

▼
Learning management systems

▼
Social and professional topics

▼
Professional topics

▼
Computing education

▼
Computing education programs

> Computer science education

> Software engineering education



Recommendations



SIGSOFT '98/FSE-6: Proceedings of the 6th ACM SIGSOFT international symposium on Foundations of software engineering

[Read More](#)

Studies on the software testing profession

ICSE '19: Proceedings of the 41st International Conference on Software Engineering: Companion Proce

This paper attempts to understand motivators and de-motivators that influence the decisions of software pro to take up and sustain software testing careers across four different countries, i.e. Canada, China, Cuba, and India. Th...

[Read More](#)

Software Developer Profession Expanding

Employment prospects for software developers are looking up. There's a greater number of developer positions globally than there are qualified software engineers to fill them. This gap will continue to increase as the number of...

[Read More](#)

Comments

DL Comment Policy

Comments should be relevant to the contents of this article, (sign in required).

Got it

0 Comments

Share

Best Newest Oldest

Nothing in this discussion yet.

Privacy

Do Not Sell My Data



**Academic Department
Membership**

Universities and colleges
can provide
ACM Professional
Membership to their
faculty at a greatly
reduced collective cost.

LEARN MORE

PDF

Help

Research and Practice



An interdisciplinary journal
that focuses on the potential
and impact of technology
on innovations and
transformation of public
institutions and governance

Learn More

dgov.acm.org



Association for
Computing Machinery

PDF

Help



PDF

Help

Brad A. Myers
Carnegie Mellon University

**A COMPREHENSIVE STUDY
OF THE MANY WAYS TO
INTERACT WITH COMPUTERS.
ESSENTIAL READING FOR
DESIGNERS CREATING
THE TECHNIQUES OF
TOMORROW!**

books.acm.org



ACM Books
Association for Computing Machinery

[View full text](#)



SIGs

Conferences

Collections

People

ACM Computing Classification System

Accessibility Statement

PDF

Help

Join

Join ACM

Join SIGs

Subscribe to Publications

Institutions and Libraries

Connect

✉ Contact us via email

f ACM on Facebook

✂ ACM DL on X

in ACM on LinkedIn

i Send Feedback

i Submit a Bug Report

The ACM Digital Library is published by the Association for Computing Machinery. Copyright © 2026 ACM, Inc.

[Terms of Usage](#) | [Privacy Policy](#) | [Code of Ethics](#)

